문자 수준 CNN과 DistilBERT 기반 캐스케이드 웹 공격 탐지의 속도–정확도 교환 분석

Speed–Accuracy Trade-off Analysis of Cascaded Character-Level CNN and DistilBERT for Web Attack Detection

요 약

인라인 웹 애플리케이션 방화벽(web application firewall, WAF)의 다중 웹 공격 탐지에서 경량 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)은 추론은 빠르나 짧은 수용 영역 때문에 세미콜론이나 파이프 등 국소 구분자를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI)을 혼동하여 위험도가 높은 CmdI 경보의 신뢰도를 떨어뜨리고, 사전학습 트랜스포머인 DistilBERT는 긴 문맥을 포착해 이 오분류를 줄이지만 단일 스트림 CPU 추론 지연이 커 실시간 인라인 배치에 부담이 된다. 본 논문은 이를 해결하기 위해 char-CNN이 대다수 요청을 즉시 판정하고 확신도가 낮은 모호한 요청만 DistilBERT로 넘기는 2단계 캐스케이드와 내부 조기 종료 구조를 구성하고, 실트래픽 기반 SR-BH 2020 데이터셋에서 단일 스레드 CPU(배치 1) 종단간 지연과 실사용자 정상 트래픽 기준으로 속도–정확도 교환을 평가한다. 실험 결과, 제안한 char-CNN → DistilBERT 캐스케이드는 전체 요청의 0.7~0.8%만 2차로 넘기면서 char-CNN 단독 대비 전체 Macro-F1(0.9884에서 0.9911로 +0.28%p, 확인 분할 +0.53%p)과 CmdI F1(0.9682에서 0.9776으로 +0.94%p, 확인 분할 +1.74%p)을 개선하고 SQLi–CmdI 공격 종류 오분류를 절반 이상 줄여 보안 관제 환경에서 신뢰할 수 있는 CmdI 경고를 제공하는 동시에, DistilBERT 단독 대비 12.3배(확인 분할 10.6배) 빠른 추론 속도를 달성하였다. 그러나 다중 분할 검증에서 2차 모델의 미세조정 편차에 따라 단독 대비 소폭의 정확도 손실 차이가 발생하고 예측 클래스별 임계값 조정이 새 분할에서 재현되지 않았으며, URL 인코딩 변형 시 1차 탐지 회피와 지연 증가에 노출됨을 확인하여 캐스케이드의 성패는 임계값 조정이 아닌 1·2차 지연 격차와 1차 모델의 변형 강건성에 달려 있음을 규명하였다.

ABSTRACT

In inline web application firewalls (WAFs), lightweight character-level convolutional neural networks (char-CNNs) offer fast inference for multi-class web attack detection, but their short receptive fields cause frequent confusion between syntactically similar SQL injection (SQLi) and command injection (CmdI), undermining the reliability of critical CmdI security alerts, whereas pre-trained transformers such as DistilBERT resolve this confusion by capturing long-range context at the cost of prohibitive single-stream CPU latency. To address this trade-off, we construct a two-stage cascade that processes most requests with char-CNN and defers only low-confidence requests to fine-tuned DistilBERT alongside an internal early-exit variant, evaluating them on the real-traffic SR-BH 2020 dataset under single-thread CPU (batch 1) end-to-end latency and real-user benign traffic. Experimental results demonstrate that the proposed char-CNN to DistilBERT cascade defers only 0.7–0.8% of requests while improving overall Macro-F1 (+0.28 pp from 0.9884 to 0.9911; +0.53 pp on the confirmation split) and CmdI F1 (+0.94 pp from 0.9682 to 0.9776; +1.74 pp on the confirmation split) over stand-alone char-CNN, cutting SQLi–CmdI attack-type confusion by more than half to enable more reliable CmdI warnings in security operations while achieving a 12.3× inference speedup (10.6× on the confirmation split) over stand-alone DistilBERT. Nevertheless, multi-seed evaluation reveals slight accuracy-loss variance relative to stand-alone DistilBERT across fine-tuning seeds, non-replication of class-wise threshold tuning on held-out seeds, and vulnerability to first-stage evasion and latency inflation under URL-encoding mutations, indicating that cascade effectiveness ultimately depends on stage latency gaps and first-stage robustness rather than threshold tuning.

Keywords: Web attack detection, Cascade inference, Character-level CNN, DistilBERT, Command injection, Inference latency

I. 서론

웹 애플리케이션 방화벽(web application firewall, WAF)은 모든 인바운드 HTTP 요청의 인라인 경로에 놓이므로, 높은 분류 정확도뿐 아니라 요청 한 건을 처리하는 지연 시간을 함께 충족해야 한다. 동시에 보안 분야의 기계학습 평가에서는 실험실 측정 조건과 실제 배포 환경의 괴리가 성능을 과장한다는 점이 반복해 지적되어 왔다[1].

본 연구는 수동 특징 공학(n-gram 빈도 사전 등)에 의존하지 않고 원시 바이트열이나 서브워드 토큰을 종단간(end-to-end) 학습하는 딥러닝 웹 공격 탐지기를 대상으로 한다. 다중 웹 공격 분류에서 보안 관제(security operations center, SOC)의 핵심 병목은 정상 대 공격의 단순 이진 차단보다, 세미콜론(;), 파이프(|), 따옴표(') 등 국소 구분자를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 사이의 공격 종류 오분류이다. 특히 운영체제 셸 명령을 직접 실행하는 CmdI는 웹 서버 장악으로 직결되는 고위험 공격이지만 전체 공격 트래픽에서 차지하는 비중은 극히 낮아(본 데이터셋 기준 전체의 2.1%, SQLi의 약 1/33), 다수 클래스인 SQLi의 일부가 CmdI로 오분류되거나 긴 CmdI 페이로드가 문맥 잘림으로 누락되면 관제 시스템의 CmdI 경보 정밀도와 신뢰도가 급격히 무너진다. 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)[2]은 단일 스레드 CPU에서 평균 0.35~0.60ms 안에 요청을 처리하지만 짧은 합성곱 창 때문에 SQLi와 CmdI를 혼동하는 한계가 남고, 사전학습 트랜스포머인 DistilBERT[3]는 전역 어텐션으로 이 혼동을 크게 줄이는 대신 요청당 약 45~46ms가 걸려 단독 인라인 배치가 부담스럽다.

이 속도–정확도 간극을 메우기 위해 가벼운 1차 모델이 대다수 요청을 즉시 판정하고 확신도가 임계값 τ 미만인 모호한 요청만 무거운 2차 모델로 넘기는 캐스케이드(cascade)[4]와 내부 조기 종료(early exit)[5]가 제안되어 왔다. 그러나 기존 웹 보안 캐스케이드 연구들에서는 네 가지 핵심 질문이 통합적으로 검증되지 않았다. 첫째, 토큰화와 바이트 인코딩을 포함한 단일 스트림(배치 1) 종단간 공정 지연에서도 char-CNN 단독 대비 F1 개선과 DistilBERT 단독 대비 대규모 가속이라는 속도–정확도 교환(trade-off)이 성립하는가? 둘째, 공격 요청이 대다수인 벤치마크 시험 집합이 아니라 낮은 실제 배포 공격 기저율(π = 3.3%)[6]과 취약점 스캐너가 아닌 실사용자 정상 트래픽 분포에서도 넘김 비용이 낮게 유지되는가? 셋째, 1차 확신도 기반의 넘김 판단(단일 임계값 및 예측 클래스별 임계값)[7]이 다중 분할 seed에서도 일관된 이득을 내는가? 넷째, 공격자가 페이로드를 변형해 2차 호출을 유도하는 비용 기반 지연 유발 공격(slowdown attack)[8]에 얼마나 노출되는가?

실트래픽 기반 다중 공격 데이터셋인 SR-BH 2020[9] 위에서 수행한 본 논문의 핵심 기여는 세 가지이다.

- **char-CNN**과 **DistilBERT** 결합 캐스케이드의 속도–정확도 교환 규명. 동일 5-fold 분할과 동일 단일 스레드 CPU에서 초경량 1차 캐스케이드(char-CNN-tiny → char-CNN), char-CNN 내부 조기 종료(char-CNN-EE), char-CNN → DistilBERT 캐스케이드, 조기 종료 출구 결합 캐스케이드(k3 exit → DistilBERT)의 네 구성을 비교하고 선행 연구(WAMM[10])의 DistilBERT를 재구현해 기준선으로 삼는다. char-CNN → DistilBERT 캐스케이드는 char-CNN 단독 대비 전체 Macro-F1을 +0.28~+0.53%p(4개 분할 전체 +0.06~+0.53%p) 높이면서 DistilBERT 단독 대비 합산 **12.3**배(확인 분할 **10.6**배; 조기 종료 출구를 1차로 결합 시 **23.8~29.3**배)의 추론 가속을 달성함을 보이고, 동시에 다중 분할 간 미세조정 편차로 인해 2차 단독 대비 사전 정확도 허용폭(0.30%p) 충족이 기본 분할(−0.03%p)과 확인 분할(−0.32%p)에서 갈리는 재현 경계를 투명하게 정량화한다.
- **SQLi–CmdI** 오분류 개선을 통한 고신뢰 **CmdI** 경보 확보 및 **128** 토큰 문맥 잘림 규명. 512 토큰 DistilBERT와 이를 2차로 둔 캐스케이드가 단지 전체 F1만 소폭 높이는 것이 아니라, 전체 요청의 0.7~0.8%만 넘기면서도 char-CNN의 핵심 약점인 SQLi–CmdI 공격 종류 오분류를 절반 이상(56~74%) 제거하고 공격 미탐(특히 CmdI → Normal 누락)까지 줄여 CmdI F1을 0.9682 → 0.9776(확인 분할 0.9618 → 0.9792), CmdI 경보 정밀도를 95.2% → 97.1%(확인 분할 93.9% → 97.3%)로 끌어올림을 밝힌다. 아울러 선행 연구의 128 토큰 제한이 긴 CmdI 요청의 26.5%를 잘라내어 오히려 char-CNN 단독보다 CmdI 탐지 성능을 크게 떨어뜨림을 입증한다.
- 실사용자 정상 트래픽 분리 평가, 넘김 판단의 구조적 한계 및 비용 공격면 규명. 단일 임계값이 공격 쪽 F1을 맞추려 임계값을 올릴 때, 1차가 정상으로 맞게 예측했으나 확신도가 덜한

실사용자 정상 요청이 대량 넘어가(특정 폴드 r_N = 33.6%) 배수를 떨어뜨리는 구조적 비용을 진단한다. 나아가 이를 완화하려던 예측 클래스별 임계값이 추가 분할 확인 실험에서 재현되지 않음(유의성 0/2)을 보이고, URL 인코딩 1회 변형으로 공격 요청당 기대 지연이 **6.4**배 증가하며 1차 탐지 회피가 발생하는 공격면을 실측한다.

결과에 맞춘 사후 기준 변경(HARKing)을 막기 위해 모든 확인 실험의 판정 규칙은 측정 전 버전 관리에 고정하였으며(커밋 2a456d7~d9d3678), 기준 외 분석은 사후 분석 또는 병기로 구분한다.

II. 관련 연구

웹 공격 탐지 연구는 HTTP 요청 문자열을 1차원 시퀀스로 다루며[11], Zhang 등[2]의 char-CNN은 별도 토크나이저 없이 원시 바이트에서 지역 n-gram 패턴을 추출하는 경량 신경망으로 널리 쓰인다. Sureda Riera 등[9]이 공개한 SR-BH 2020은 실운영 웹 서버 앞단의 ModSecurity 로그를 검수해 CAPEC 공격 분류를 붙인 다중 클래스 데이터셋이다. 이 데이터셋 위에서 Osama 등[10]은 중복 제거와 정제를 거친 WAMM 프레임워크를 통해 DistilBERT[3]의 성능을 보고하였고, 이중 채널 BERT(WADBERT)[12]와 라벨 정제 기반 이상 탐지(X-WAD)[13]가 후속 연구로 제안되었다. 본 연구는 Osama 등[10]의 딥러닝 최고 모델인 DistilBERT를 동일 분할·동일 CPU에서 512 토큰 및 원 설정(128 토큰)으로 재구현해 직접 비교한다.

캐스케이드 추론은 최대 소프트맥스 확률(maximum softmax probability, MSP)[4][14] 임계값으로 경량 1차와 대형 2차를 잇는 구조로, Tasdemir 등[6]이 SQLi 단일 공격 탐지에서 BERT 대비 약 20배 가속을 보고하였다. 어느 요청을 넘길지 결정하는 문제는 선택적 분류[15] 및 넘김 학습[16]과 맞닿아 있는데, Enomoto와 Eda[17]는 온도 보정[18]이 캐스케이드 효율을 떨어뜨릴 수 있음을 보였고, Jitkrittum 등[7]은 최적 넘김 규칙이 1·2차 정답 확률 차이에 의해 결정되므로 2차가 특정 하위 분포에만 강할 때 1차 확신도 게이트가 구조적으로 한계를 가짐을 증명하였다. 단일 신경망 내부에 보조 분류기를 두는 조기 종료(BranchyNet)[5]는 전처리 중복을 없앤 대안이다. 한편 적응형 추론은 조기 종료를 방해해 연산 비용을 폭증시키는 지연 유발 공격(DeepSloth)[8]에 노출되며, 인라인 보안 장비 평가에서는 단일 스트림 지연 분리 측정(MLPerf)[19]과 낮은 실제 공격 기저율(base rate)[20] 반영이 필수적이다.

III. 방법

3.1 입력 표현과 딥러닝 캐스케이드 구성

각 HTTP 요청에서 URI와 본문(body)을 이어 붙인 원시 바이트열을 $b = (b_0, b_1, ..., b_{L-1})$이라 한다. 문자 수준 모델(**char-CNN**[2] 계열)은 고정 길이 패딩 없이 최대 2,304바이트 상한 안에서 실제 길이 L 그대로의 1차원 바이트 시퀀스를 읽는다(본 데이터셋의 URI+본문 길이는 중앙값 96바이트, 99백분위수 352바이트, 최댓값 674바이트여서 잘리는 요청이 0건이며, 최대 커널 크기인 7바이트보다 짧은 경우만 7바이트까지 채운다). 각 바이트(257개 기호)를 임베딩한 뒤 1차원 합성곱과 시간축 최대 풀링을 수행해 선형 분류기에 넣는다. 사전학습 트랜스포머인 **DistilBERT**[3]는 원문 문자열을 WordPiece 토크나이저(소문자화 포함)로 분절해 입력받는다.

본 연구는 경량 기준선인 **char-CNN**(임베딩 64차원, 커널 3·5·7 각 128채널 병렬, 141,252 파라미터)과 고비용 참조 모델인 **DistilBERT**(`distilbert-base-uncased`, 66.96M 파라미터) 사이에서 속도–정확도 교환을 달성하기 위해 네 가지 딥러닝 구성을 설계·비교한다(Table 1).

1. 초경량 **1**차 캐스케이드(`char-CNN-tiny` → `char-CNN`): 임베딩을 32차원으로 줄이고 커널 3 분기 하나(64채널, 14,692 파라미터)만 남긴 초경량 모델을 1차로, 전체 char-CNN을 2차로 잇는 캐스케이드이다.
2. 내부 조기 종료(`char-CNN-EE`): 별도 1차 모델 없이 char-CNN의 커널 3 분기 풀링 출력(128차원) 뒤에 보조 선형 분류기(128→4)를 추가한 구조이다(141,768 파라미터). 학습 시 주 헤드 손실과 보조 헤드 손실을 $L = L_{main} + 0.3\ L_{aux}$로 결합해 동시에 최적화한다. 추론 시에는 임베딩과 커널 3 분기만 먼저 계산해 보조 헤드 확신도가 임계값 이상이면 즉시 종료하고, 미달할 때만 커널 5·7 분기를 마저 계산한다. 보조 헤드 경로와 동일한 크기(임베딩 64, 커널 3 하나, 128채널, 41,668 파라미터)를 처음부터 혼자 학습한 대조 모델을 **char-CNN-k3**라 한다.

3. 주 캐스케이드(char-CNN → DistilBERT): 전체 char-CNN을 1차로 내리고 미세조정된 DistilBERT를 2차로 두는 2단계 캐스케이드이다. 선행 연구(WAMM[10])는 최대 길이를 128 토큰으로 두었으나, 본 데이터셋의 학습+검증 풀(226,048행)에서 128 토큰 제한은 전체의 6.9%(15,553행), 특히 길이가 긴 CodeInjection의 24.7%와 CmdI의 26.5%를 잘라내어 참조 모델에 불리하게 작용한다. 따라서 주 실험에서는 잘림이 0%인 **512** 토큰(토큰 수 중앙값 54, 99백분위수 189)을 사용해 참조 모델의 상한 성능을 보존하고, 선행 연구의 **128** 토큰 설정(DistilBERT-128)은 5.3절에서 별도로 대조한다.
4. 조기 종료 출구 결합 캐스케이드(k3 exit → DistilBERT): char-CNN-EE에서 결합 학습된 커널 3 보조 헤드 출구만 떼어내어 1차로 삼고, DistilBERT(512 토큰)를 2차로 연결해 1차 추론 비용까지 줄인 결합 캐스케이드이다.

Table 1. Deep-learning models compared in 5-fold cross-validation

Model	Role & setup	Input	Params
char-CNN	Baseline ref.; 2nd stage of tiny cascade; 1st stage of main cascade	raw bytes (L)	141,252
char-CNN-tiny	1st stage of tiny cascade (k3, 64-ch)	raw bytes (L)	14,692
char-CNN-k3	Single-branch stand-alone control (k3, 128-ch)	raw bytes (L)	41,668
char-CNN-EE	Joint-trained internal early exit; 1st stage of k3-exit cascade	raw bytes (L)	141,768
DistilBERT	High-accuracy ref. (WAMM[10]); 2nd stage of DistilBERT cascades	≤512 (or 128) tok.	66.96M

Notes: All models use class-weighted cross-entropy on the same 5 folds (early stopping patience 5 on internal validation Macro-F1; DistilBERT fine-tuned for 3 epochs with Adam, lr 2e-5, batch 16).

3.2 넘김 판단 규칙과 배포 기저율 재가중 합산 배수

1차 모델 h_1이 요청 x에 대해 출력한 소프트맥스 확률을 $p_1(k|x)$, 최대 소프트맥스 확률을 $s(x) = \max_k p_1(k|x)$라 할 때, 임계값 τ에 따른 2단계 캐스케이드의 최종 예측 $\hat{y}(x)$는 다음과 같다.

$$\hat{y}(x) = h_1(x) \text{ if } s(x) \geq \tau, \text{ otherwise } h_2(x) \quad (1)$$

기본 단일 임계값**(single threshold)** 방식은 각 폴드의 내부 검증 집합에서 검증 Macro-F1이 2차 단독 검증 Macro-F1의 0.5%p 이내를 유지하는 지점 중 배포 기대 넘김률(식 3)이 최소인 τ를 골라 시험 폴드에 1회 적용한다. 이에 대조하는 예측 클래스별 임계값**(class-wise threshold)** 방식은 클래스마다 확신도 분포와 오분류 비용이 다른 점을 반영해, 1차 예측 클래스 $\hat{c}_1(x) = \text{argmax}_k\ p_1(k|x)$별로 서로 다른 임계값 τ_c를 둔다.

$$\text{defer } x \Leftrightarrow s(x) < \tau_{\hat{c}_1(x)},\ \hat{c}_1(x) = \text{argmax}_k\ p_1(k|x) \quad (2)$$

임계값 벡터 $(\tau_1, ..., \tau_4)$는 내부 검증 집합에서 전체 넘김 예산을 0.5%p씩 늘릴 때 단위 넘김당 Macro-F1 상승폭이 가장 큰 클래스에 예산을 배정하는 탐욕적 경로 탐색으로 정한다.

1차 지연을 t_1, 2차 지연을 t_2, 비교 기준 단독 모델 지연을 t_{ref}라 하자(char-CNN-EE는 앞단 커널 3 출구 시간을 t_{front}, 뒷단 커널 5·7 시간을 t_{rest}라 할 때 $t_1 = t_{front}$, $t_2 = t_{rest}$, t_{ref} = char-CNN 지연; DistilBERT 결합 캐스케이드는 t_{ref} = DistilBERT 지연 t_R). 넘김률이 정상 요청(r_N)과 공격 요청(r_A)에서 크게 다르므로, 배포 환경의 공격 기저율 $\boldsymbol{\pi = 3.3\%}$[6]를 반영한 기대 넘김률 E_π와 폴드 f의 기대 지연 t_f는 다음과 같다.

$$E_\pi = \pi \cdot r_A + (1 - \pi) \cdot r_N,\ t_f = t_1 + E_\pi \cdot t_2 \quad (3)$$

교차검증의 5개 폴드에서 폴드별 속도 배수 $S_f = t_{ref} / t_f$를 단순 산술평균하면, 넘김이 많아 느려진 폴드의 비용이 가려져 배포 속도를 과대평가하게 된다. 따라서 본 논문은 다섯 폴드의 평균 비용으로 계산한 합산 속도 배수 $\bar{S}$(폴드 배수 S_f의 조화평균)를 헤드라인 지표로 쓰고 산술평균은 괄호에 병기한다.

$$\bar{S} = t_{ref} / \left((1/5) \Sigma_f t_f \right) \quad (4)$$

IV. 실험 환경

4.1 데이터셋과 실사용자 정상 층 분리

SR-BH 2020[9] 원본 907,815행의 CAPEC 라벨을 정상(Normal), SQL 삽입(SQLi), 코드 삽입(CodeInjection, 서버 측 PHP 실행 탐침 위주), 명령 삽입(CmdI)의 4클래스로 매핑하였다. 정규표현식 감사로 적발된 정상 라벨 내 공격 구문 의심 행(30,280건), 동일 문자열 다중 라벨 충돌 행, 완전 중복 행을 제거해 총 **265,939**행(학습 186,157 · 검증 39,891 · 봉인 시험 39,891; 공격 비율 74.8%)을 구성하였다. 입력은 URI와 본문만 사용한다(쿠키 헤더를 추가한 구성은 쿠키 값을 마스킹한 대조군과 보정 $p = 0.17$로 유의한 차이가 없어 세션 식별자 과적합 가능성을 배제함). 최소 클래스인 CmdI는 전체 265,939행 중 **5,551**건**(2.1%)**, 5-fold 교차검증에 사용하는 학습+검증 풀(226,048행) 기준 **4,718**건**(**학습 **3,886**건·검증 **832**건**, 2.1%)**으로 다수 공격 클래스인 SQLi(학습+검증 풀 156,039건, 69.0%)의 약 1/33에 불과하다.

데이터 분석에서 확인된 핵심 특성은 정상(Normal) 요청의 출처 편향이다. 학습+검증 풀의 정상 요청 56,906건(전체 데이터셋 66,948건)을 원본 로그의 User-Agent 출처별로 층화 분석한 결과, 취약점 스캐너(Nikto, OpenVAS 등)에서만 관찰된 요청이 **96.0%(54,600**건**;** 전체 **64,250**건**)**, 스캐너와 일반 브라우저에서 중복 관찰된 요청이 **3.1%(1,756**건**;** 전체 **2,053**건**)**를 차지해 스캐너 유래 텍스트가 99.0%에 달했으며(단일 최빈 스캐너 User-Agent 행 기준 97.8%), 스캐너 없이 일반 브라우저 User-Agent로만 들어온 순수 실사용자 정상 요청은 학습+검증 풀 내 **550**건**(1.0%,** 폴드당 약 **110**건**;** 전체 데이터셋 **645**건**)**뿐이었다. 실제로 동일 5-fold 교차검증에서 char-CNN 기준 실사용자 정상 층의 오탐률(false positive rate, FPR)은 **2.00%(seed 42; seed 43**은 **1.27%)**로 스캐너 정상 층**(0.35%**, seed 43은 0.24%)의 **5.7**배**(seed 43**은 **5.3**배**)**였으며, char-CNN → DistilBERT 캐스케이드의 정상 넘김률 r_N 또한 스캐너 위주 검증 분포에서는 0.86%(seed 43은 0.79%)에 불과했으나 실사용자 정상 층에서는 평균 **7.08%(**약 **8.2**배**,** 특정 폴드 최대 **33.6%; seed 43**은 평균 **8.48%**로 약 **10.7**배**)**로 급증하였다. 따라서 스캐너 위주의 전체 정상 지표로 평가하면 오탐과 넘김 비용을 크게 과소평가하게 되므로, 모든 평가에서 실사용자 정상 층과 스캐너 층을 분리하고 주 판정은 실사용자 정상 $\mathbf{r_N}$ 기준으로 수행한다.

4.2 평가 절차와 사전 고정 판정 기준

다중 클래스 불균형을 반영해 주 분류 지표는 클래스별 F1의 비가중 평균인 **Macro-F1**을 사용한다. 모든 실험은 봉인된 시험 집합(39,891행)을 제외한 학습+검증 풀 **226,048**행의 동일한 층화 5-fold 교차검증에서 수행한다(각 폴드 내부 학습 부분을 9:1로 다시 나누어 조기 종료 및 임계값 τ 선택용 내부 검증 집합으로 사용). 폴드 간 차이는 학습 집합 중복을 교정하는 Nadeau–Bengio 보정 대응 t-검정[21]과 Holm 보정으로 검정한다. 분할 및 초기화 seed는 42를 기본 판정에, 43(클래스별 임계값 추가 확인은 44·45)을 사전 등록된 재현성 확인에 사용한다.

추론 지연은 단일 데스크톱(AMD Ryzen 7 9800X3D, RAM 32GB, RTX 4080 SUPER, Windows 11, PyTorch 2.6.0)에서 인라인 WAF의 단일 스트림 처리를 반영하여 **CPU** 단일 스레드**(1 thread)**·배치 **1** 조건에서 원문 입력부터 전처리(바이트 인코딩·토큰화)와 순전파를 거쳐 예측이 나오기까지의 종단간 시간을 잰다(2,000건 × 20회 워밍업 후 30회 반복 중앙값 p50)[19]. 연속 실행 간 클럭 표류(4~7%)와 측정 시점별 OS 배경 상태에 따른 절대 지연 변동(예: 서로 다른 측정 시점 간 char-CNN 중앙값 0.345ms 대 0.563~0.604ms)을 엄격히 통제하기 위해, 비교 대상 모델들은 반드시 같은 실행 안에서 요청마다 번갈아 측정해 동일 실행 내 측정값끼리만 속도 배수를 계산한다.

기준 단독 모델 대비 우위("위") 사전 판정 기준은 측정 전에 다음 세 조건을 모두 충족하는 것으로 고정하였다(실사용자 정상, $\pi = 3.3\%$): ① 합산 배수 $\bar{\mathbf{S}} \geq \mathbf{1.2}$**배,** ② **5**개 폴드 모두 $\mathbf{S_f > 1.0}$**배,** ③ 기준 모델 대비 **Macro-F1** 평균 손실 $\leq \mathbf{0.30\%p}$**(**$\mathbf{\Delta F1 \geq -0.30\%p}$**)**.

V. 실험 결과

5.1 딥러닝 캐스케이드의 속도–정확도 교환: char-CNN 대비 F1 개선과 DistilBERT 대비 12배 가속

본 학습에 앞서 수행한 층별 지연 분해 예비 측정에서 char-CNN 추론 시간의 **91.0%**가 세 합성곱 층**(**커널 **3·5·7)**에서 소비되고 바이트 인코딩과 호출 고정비는 **8.5%**에 불과함이 확인되어, 커널을 줄이거나 앞 분기만 실행하는 구조의 물리적 가속 가능성을 확인하였다. 또한 2차 모델인 재구현 DistilBERT(512 토큰) 단독은 5-fold 교차검증에서 Macro-F1 **0.9914**(seed 42) 및 **0.9947**(seed 43)을 기록해 char-CNN 단독**(0.9884** 및 **0.9862**)을 일관되게

상회하였다. Table 2와 Fig. 1은 네 가지 딥러닝 구성의
본 측정 및 확인 실험 결과를 종합한 것이다.

Table 2. Speed–accuracy trade-off of deep-learning cascades vs. stand-alone references (5-fold CV, CPU batch 1, 1 thread, end-to-end unpadded latency, real-user benign traffic at π = 3.3%)

Configuration	Reference R (F1)	Seed	Threshold rule	Macro-F1 (ΔR / Δchar-CNN)	r_N / r_A	Pooled $\bar{S}$ (arith.*)	Fold S_f range	Verdict vs. R
char-CNN-tiny → char-CNN	char-CNN (0.9884)	42	Single τ	0.9839 (−0.45 pp / −0.45 pp)	30.2% / 4.0%	2.13× (2.39×*)	1.52–4.10×	No (F1 loss)
char-CNN-EE (internal early exit)	char-CNN (0.9884)	42	Single τ	0.9874 (−0.10 pp / −0.10 pp)	1.1% / 0.4%	**3.52×** (3.53×*)	3.24–3.62×	**Yes**
char-CNN-EE (confirm.)	char-CNN (0.9862)	43	Single τ	0.9895 (+0.33 pp / +0.33 pp)	0.0% / 0.005%	**3.62×** (3.62×*)	3.62–3.62×	**Yes (2/2)**
Control: char-CNN-k3 (solo)	char-CNN (0.9884)	42	—	0.9748 (−1.36 pp / −1.36 pp)	0.0% / 0.0%	3.62×	3.62–3.62×	No (F1 loss)
char-CNN solo vs. DistilBERT	DistilBERT (0.9914)	42	—	0.9884 (−0.303 pp / 0.00 pp)	0.0% / 0.0%	80.2×	80.2–80.2×	No (0.003 pp short)
char-CNN → DistilBERT	DistilBERT (0.9914)	42	Single τ	**0.9911 (−0.03 pp / +0.28 pp)**	7.1% / 0.7%	**12.3×** (51.6×*)	2.95–80.2×	**Yes (seed 42)**
char-CNN → DistilBERT (confirm.)	DistilBERT (0.9947)	43	Single τ	**0.9914 (−0.32 pp / +0.53 pp)**	8.5% / 0.7%	**10.6×** (38.3×*)	3.93–80.2×	No (p = 0.038 vs. R)
char-CNN → DistilBERT (class-wise τ)	DistilBERT (0.9914)	42	Class-wise τ	0.9920 (+0.06 pp / +0.37 pp)	0.7% / 0.7%	51.1× (57.5×*)	32.5–80.2×	Secondary (failed on s44–45)
char-CNN → DistilBERT (class-wise τ)	DistilBERT (0.9947)	43	Class-wise τ	0.9920 (−0.27 pp / +0.58 pp)	1.1% / 0.6%	42.5× (50.6×*)	13.7–80.2×	Secondary (failed on s44–45)
k3 exit → DistilBERT	Main cascade (12.2×)	42	Single τ	0.9883 (−0.28 pp / −0.00 pp)	3.2% / 0.4%	**29.3×** (86.8×*, ×2.40)	15.0–289.8×	**Yes (vs. main cascade)**
k3 exit → DistilBERT (confirm.)	Main cascade (10.5×)	43	Single τ	0.9919 (+0.05 pp / +0.57 pp)	4.0% / 0.4%	**23.8×** (124.9×*, ×2.27)	12.1–289.8×	**Yes (2/2 vs. main cascade)**

Notes: Values with * are arithmetic means of fold speedups shown in parentheses. Same-run p50 latencies (never mixed across runs): char-CNN cascade run has char-CNN-tiny t_1 = 0.0606 ms, char-CNN t_2 = 0.3451 ms, k3 front t_{front} = 0.0954 ms; main DistilBERT cascade run (seeds 42–43) has char-CNN t_2 = 0.563 ms, DistilBERT(512) t_R = 45.15 ms (80.2×); token-comparison and k3-exit run (seeds 44–45 and k3-exit cascade) has t_2 = 0.604 ms, t_{front} = 0.159 ms, $t_R(512)$ = 46.10 ms, $t_R(128)$ = 46.08 ms.

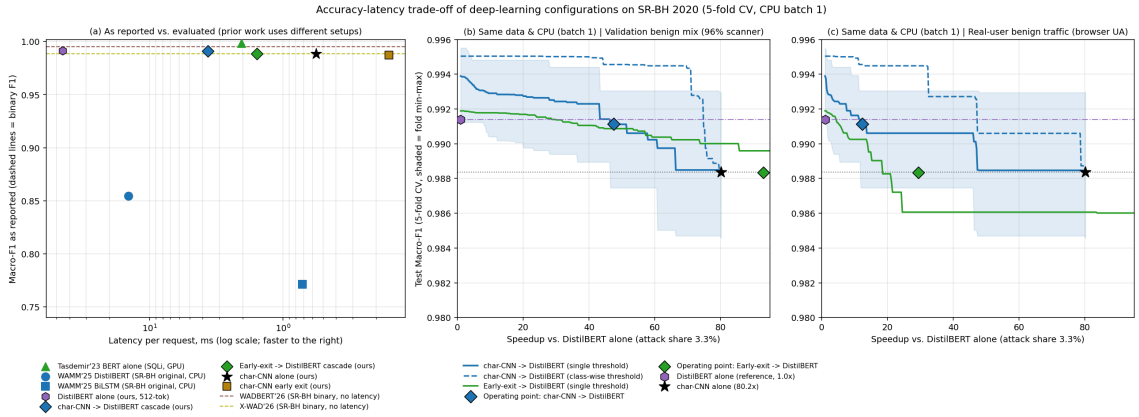


Fig. 1. Accuracy–latency trade-off of deep-learning configurations on SR-BH 2020 (5-fold CV, CPU batch 1, 1 thread). Panel (a) shows end-to-end latency alongside reference points reported by prior work under different hardware/class setups; panels (b) and (c) compare speedup vs. stand-alone DistilBERT against 5-fold Macro-F1 on the exact same folds under validation benign mix (96% scanner) and real-user benign traffic.

1) 초경량 **1**차 캐스케이드(char-CNN-tiny → char-CNN)**: 1**차 용량 부족으로 인한 탈락**.** 초경량 1차(t_1 = 0.0606ms)는 단독 Macro-F1이 0.9601로 낮아 실사용자 정상 요청의 30.2%를 넘겼고, 합산 배수는 **2.13**배(산술 2.39배)에 그쳤으며 Macro-F1이 0.9839로 char-CNN 단독 대비 **−0.45%p** 하락해 사전 기준(손실 0.30%p 이내)을 넘지 못했다.

2) 내부 조기 종료(char-CNN-EE)**:** 넘김 판단이 아닌 결합 학습 기반 모델 압축 효과**.** char-CNN 내부의 커널 3 분기에 보조 헤드를 결합 학습한 char-CNN-EE는 seed 42에서 합산 **3.52**배(산술 3.53배, F1 0.9874, 단독 대비 −0.10%p), 확인용 seed 43에서 합산 **3.62**배(F1 0.9895, 단독 대비 +0.33%p)를 기록해 두 seed 모두 char-CNN 단독과 동등한 정확도를 **3.5~3.6**배 빠르게 내며 사전 기준을 통과하였다. 그러나 폴드별 운영점을 보면 seed 42의 다섯 폴드 중 셋, seed 43의 전 폴드가 넘김률 **0%**였다(보조 헤드 단독 F1이 이미 0.9849). 즉 이득의 원천은 넘김 판단이 아니라 결합 학습된 보조 헤드 자체였다. 같은 크기를 처음부터 혼자 학습한 대조 모델 char-CNN-k3(seed 42, F1 **0.9748**, char-CNN 대비 −1.36%p)는 보조 헤드보다 1.01%p 낮아 기준을 통과하지 못했다(보정 p = 0.34; 최적 에폭은 seed 43 보조 헤드가 12~19였고, 같은 seed 42에서 비교하면 보조 헤드는 8·10·2·19·3, k3 단독은 3·20·9·6·2였다). 조기 종료 인내(patience)를 늘린 추가 대조 실험은 수행하지 않았으므로, +1.01%p 차이가 결합 학습의 표현력 몫인지 단독 모델의 이른 조기 종료(최적화 안정성) 인지는 가리지 못했다. 따라서 내부 조기 종료의 이득은 넘김 판단 효과가 아닌 결합 학습을 통한 모델 압축**(k3** 보조 출구**)**으로 해석한다.

3) 주 캐스케이드(char-CNN → DistilBERT)**: char-CNN** 대비 **F1** 개선과 **DistilBERT** 대비 **12**배 가속의 교환**.** 동일 실행에서 char-CNN(0.563ms)과 DistilBERT(45.15ms)의 지연 격차는 **80.2**배였다. 가장 빠른 끝단인 char-CNN 단독은 80.2배 빠르나 DistilBERT 대비 F1이 seed 42에서 −0.303%p(0.9884 대 0.9914), seed 43에서 −0.85%p(0.9862 대 0.9947) 낮았고 특히 소수 클래스인 CmdI에서 오분류가 집중되었다. 반면 char-CNN → DistilBERT 캐스케이드(단일 임계값)는 공격 요청의 0.7%, 실사용자 정상 요청의 7.1~8.5%(전체 시험 요청의 0.7~0.8%)만 2차로 넘기면서 두 가지 핵심 이점을 동시에 달성하였다. 첫째, **char-CNN** 단독 대비 전체 **Macro-F1**을 **seed 42**에서 **+0.28%p(0.9884 → 0.9911), seed 43**에서 **+0.53%p(0.9862 → 0.9914)** 끌어올렸고, 5.2절의 추가 확인 분할(seed 44·45)을 포함한 4개 분할 모두에서 일관된 양(+)의 개선(+0.06~+0.53%p)을 기록하였다(단, 전체 4클래스 평균에 대한 5-fold 보정 t-검정에서는 p = 0.34·0.23으로, 이득이 소수 클래스인 CmdI F1 [+0.94%p, +1.74%p]에 집중됨을 5.3절에서 구체적으로 입증한다). 둘째, **DistilBERT** 단독 대비 실사용자 정상 기준 합산 배수로 **seed 42**에서 **12.3**배(산술 **51.6**배, 평균 지연 **3.67ms), seed 43**에서 **10.6**배(산술 **38.3**배, 평균 지연 **4.28ms)** 빨라, 45ms대의 무거운 트랜스포머 지연을 3.7~4.3ms대로 단축하였다. 한편 DistilBERT 단독 대비 Macro-F1 손실은 **seed 42**에서 **−0.03%p(0.9911** 대 **0.9914), seed 43**에서 **−0.32%p(0.9914** 대 **0.9947,** 보정 **p = 0.038)**

였다. 즉 2차 단독 대비 사전 허용 기준(손실 0.30%p 이내)은 **seed 42**에서는 충족했으나 확인용 seed 43에서는 0.02%p 차로 미달하였다. 그 원인을 보면 캐스케이드 자체의 Macro-F1은 0.9911 → 0.9914로 오히려 소폭 올랐으나, 기준점인 DistilBERT 단독이 3에폭 미세조정의 seed 간 편차로 0.9914 → 0.9947로 상승했기 때문이다. 캐스케이드는 트래픽의 약 93%(실사용자 정상 기준)를 1차 char-CNN 판정으로 끝내므로 2차 단독이 특정 seed에서 더 강해져도 1차가 높은 확신도로 틀린 소수 표본이 상한을 묶는다.

4) 조기 종료 출구 결합 캐스케이드**(**k3 exit → DistilBERT**): 1**차 경량화를 통한 **24~29**배 가속**.** 주 캐스케이드의 1차를 char-CNN 전체(0.604ms) 대신 결합 학습한 k3 보조 출구(0.159ms)로 바꾸면, 같은 2차(DistilBERT, 46.10ms)에서 합산 배수가 **seed 42**에서 **29.3**배**(**주 캐스케이드 **12.2**배 대비 **×2.40, F1 0.9883,** 주 캐스케이드 대비 **−0.28%p**), **seed 43**에서 **23.8**배**(**주 캐스케이드 **10.5**배 대비 **×2.27, F1 0.9919,** 주 캐스케이드 대비 **+0.05%p)**로 커져 두 seed 모두 주 캐스케이드 대비 사전 기준을 충족하였다. 단, seed 42의 −0.28%p는 허용 경계이고 최대 289.8배 폴드는 넘김이 0%여서 가속의 상당 부분은 1차 자체가 약 4배 가벼워진 것(0.159ms 대 0.604ms)에서 비롯된다.

5.2 넘김 판단의 한계: 느린 폴드 진단과 예측 클래스별 임계값 재확인

단일 임계값의 구조적 비용**(**사후 진단**).** seed 42의 char-CNN → DistilBERT에서 합산 배수(12.3배)가 산술평균(51.6배)보다 낮았던 원인은 폴드 4의 배수가 **2.95**배(나머지 네 폴드는 46.9~80.2배)였기 때문이다. 폴드 4에서는 검증 F1 목표를 맞추려 검증 넘김 예산이 3.0%(τ = 0.985)까지 올라갔고, 이때 실사용자 정상 넘김률 r_N이 **33.6%**(Wilson 95% 신뢰구간 [25.4%, 43.0%], n = 107; 내부 검증에서도 28.9%)로 커졌다. 넘어간 실사용자 정상 36건 중 **34**건은 **1**차가 이미 **Normal**로 맞게 예측했으나 확신도가 임계값보다 약간 낮은 요청이었다. 즉 단일 임계값은 공격 분류 F1을 높이려 임계값을 올리면 실사용자 정상 요청이 대량으로 함께 넘어가는 구조적 비용을 안고 있다.

예측 클래스별 임계값의 추가 분할**(seed 44·45)** 확인 실패**.** 이 구조적 비용 때문에 seed 42·43에서 예측 클래스별 임계값을 적용했을 때는 Normal 예측 클래스의 임계값만 낮게 유지되어 실사용자 정상 넘김이 억제되었고, 합산 배수가 51.1배·42.5배로 크게 뛰었다(Table 2). 이 이득이 새로운 분할에서도 일반화되는지 검증하기 위해 측정 전 사전 등록한 추가 분할**(seed 44·45)** 확인 실험(채택 조건: ① 단일 임계값 대비 합산 배수 이득 ≥ 20%, ② Macro-F1 손실 ≤ 0.30%p, ③ 폴드별 기대 넘김률 E_π 차이 보정 $p < 0.05$)을 수행한 결과, 두 **seed** 모두 세 조건을 동시에 채우지 못했다.

- **seed 44**: 단일 임계값 합산 배수 **76.0**배 → 클래스별 임계값 **59.6**배**(−21.6%,** 조건 ① 미달**)**, Macro-F1 0.9908 → 0.9920(char-CNN 단독 0.9901, DistilBERT 단독 0.9901; 조건 ② 충족), E_π 차이 보정 p = **0.34(**조건 ③ 미달**)**.
- **seed 45**: 단일 임계값 합산 배수 **42.4**배 → 클래스별 임계값 **54.4**배**(+28.4%,** 조건 ① 충족**)**, Macro-F1 0.9903 → 0.9913(char-CNN 단독 0.9897, DistilBERT 단독 0.9927; 조건 ② 충족), E_π 차이 보정 p = **0.54(**조건 ③ 미달**)**.

원인을 분석하면, 클래스별 임계값의 속도 이득은 단일 임계값이 실사용자 정상을 많이 넘기는 느린 운영점을 고른 특정 폴드(seed 45 폴드 1: 17.6→32.5배, seed 42 폴드 4와 같은 유형)를 메우는 데서만 발생하였다. 반면 seed 44에서는 단일 임계값이 다섯 폴드 모두에서 이미 상한(76.0배, r_N = 0.0%, r_A = 0.1%)에 도달해 클래스별 임계값이 줄일 비용이 없었고 오히려 두 폴드의 속도를 깎았다. 즉 seed 42·43에서 관찰한 클래스별 임계값의 큰 속도 이득은 새 seed 44·45에서 재현되지 않았으며(속도 기준 1/2 충족, 통계적 유의성 0/2), 넘김 임계값 규칙을 정교화하는 시도는 일관된 이득을 보장하지 못했다. (한편 이 확인 실험에서 char-CNN → DistilBERT 단일 임계값은 char-CNN 단독 대비 seed 44 +0.07%p, seed 45 +0.06%p 높았고, 동일 seed DistilBERT 단독 대비로는 seed 44 +0.08%p, seed 45 −0.24%p여서 네 개 seed[42~45] 중 세 개 seed에서 2차 단독 대비 손실 0.30%p 이내를 만족하였다.)

5.3 SQLi–Cmdl 오분류 개선을 통한 유효한 Cmdl 경고 확보와 128 토큰 문맥 잘림 검증

char-CNN보다 80배 무거운 DistilBERT와 이를 결합한 캐스케이드가 실제로 어떤 보안 관제 가치를 만들어내는지 규명하기 위해, 5-fold 합산 226,048행 전체에 대한 혼동행렬 정밀 분해와 선행 연구(WAMM[10])의 128 토큰 설정 재구현 실험을 수행하였다(Table 3).

Table 3. Pooled 5-fold error decomposition (226,048 rows; True SQLi = 156,039, True CmdI = 4,718) comparing stand-alone char-CNN, the proposed char-CNN → DistilBERT cascade (single threshold), stand-alone DistilBERT (512 tokens), and the 128-token DistilBERT setup

Metric / error count	s42 char-CNN	s42 Cascade (ours)	s42 DistilBERT(512)	s42 DistilBERT(128)	s43 char-CNN	s43 Cascade (ours)	s43 DistilBERT(512)	s43 DistilBERT(128)
Overall Macro-F1	0.9884	**0.9911**	0.9914	0.9718	0.9862	**0.9914**	0.9947	0.9777
Missed attack (Attack → Normal)	791	**697 (−11.9%)**	736 (−7.0%)	773	839	**743 (−11.4%)**	777 (−7.4%)	826
└ Missed CmdI (CmdI → Normal)	28	**25**	23	31	30	**25**	23	67
Benign false alarm (Normal → Attack)	206	247	245	253	140	164	118	194
Real-user / scanner FPR	2.00% / 0.35%	2.00% / 0.42%	1.82% / 0.43%	2.00% / 0.43%	1.27% / 0.24%	1.27% / 0.28%	1.09% / 0.21%	1.27% / 0.33%
Attack-type confusion (total)	**266**	**196 (−26.3%)**	**167 (−37.2%)**	**934**	**359**	**220 (−38.7%)**	**59 (−83.6%)**	**629**
└ SQLi → CmdI false alerts	**192**	**123 (−35.9%)**	**99 (−48.4%)**	**847**	**252**	**113 (−55.2%)**	**2 (−99.2%)**	**551**
CmdI Precision / Recall (%)	95.18 / 98.52	**97.07 / 98.45**	97.26 / 98.41	83.52 / 98.47	93.90 / 98.58	**97.32 / 98.52**	99.64 / 98.43	88.24 / 97.84
CommandInjection (CmdI) F1	**0.9682**	**0.9776 (+0.94 pp)**	**0.9784 (+1.02 pp)**	**0.9038**	**0.9618**	**0.9792 (+1.74 pp)**	**0.9903 (+2.85 pp)**	**0.9284**

1) SQLi–CmdI 오분류 개선과 보안 관제**(SOC)**에서의 유효한 **CmdI** 경고 확보**.** Table 3에서 확인되듯, 공격을 정상으로 놓친 전체 미탐(Attack → Normal)은 char-CNN 단독에서 DistilBERT(512 토큰) 단독으로 갈 때 두 seed 모두 약 **7%**(791→736건, 839→777건) 감소했고, 오히려 제안한 char-CNN → DistilBERT 캐스케이드에서 **697**건**(−11.9%, seed 42)·743**건**(−11.4%, seed 43)**으로 가장 적었으며, 실제 CmdI 공격을 정상으로 놓친 미탐(CmdI → Normal) 또한 char-CNN 단독의 28건·30건에서 캐스케이드 **25**건**·25**건으로 줄어들었다(정답 탐지된 CmdI는 4,718건 중 4,645건·4,647건). 실사용자 정상 FPR(n = 550) 역시 2.00% 대 1.82%(seed 42), 1.27% 대 1.09%(seed 43)로 요청 1건 차이에 불과했다. 즉 2차 트랜스포머 결합의 실질적 가치는 단순 이진 차단이 아니라 **SQLi**와 **CmdI** 사이의 공격 종류 오분류 해결에 있다. 본 데이터셋(학습+검증 풀 226,048행)에서 다수 공격인 SQLi(156,039건, 69.0%)는 소수 고위험 공격인 CmdI(4,718건, 2.1%)보다 33배 많기 때문에, char-CNN이 세미콜론(;), 파이프(|), 따옴표(') 등 국소 구분자만 보고 SQLi나 정상 요청의 일부를 CmdI로 오분류하면(seed 42에서 SQLi→CmdI 192건·Normal→CmdI 38건, seed 43에서 SQLi→CmdI 252건·Normal→CmdI 50건) 관제 시스템의 CmdI 경보 대기열이 거짓 경보로 오염되어 **CmdI** 경보 정밀도**(Precision)**가 **95.18%(seed 42)·93.90%(seed 43)**까지 떨어진다(반면 실제 CmdI가 SQLi로 빠져나가는 건수는 char-CNN 46건·36건, 캐스케이드 48건·45건으로 전체 CmdI의 1% 미만에 머문다). 전체 시험 요청에서 1차 char-CNN 확신도가 0.99 이상인 비율은 97.0%이지만, DistilBERT가 교정한 오답 집합에서 char-CNN 확신도 중앙값은 **0.775(seed 42)·0.766(seed 43)**으로 낮았고 확신도 ≥ 0.99 비율은 **15.1%·10.7%**에 그쳤다. 이 구조적 특성 덕분에 char-CNN → DistilBERT 캐스케이드는 전체 시험 요청의 **0.7~0.8%**만 **2**차로 넘기면서도 **SQLi→CmdI** 오분류를 **seed 42**에서 **192→123**건**(**단독 **99**건까지 감소분의 **74%** 회수**), seed 43**에서 **252→113**건**(**단독 **2**건까지 감소분의 **56%** 회수**)**으로 대폭 줄이고 **Normal→CmdI** 오탐도 절반으로 낮추었다**.** 그 결과 캐스케이드의 **CmdI** 경보 정밀도는 **97.07% (+1.89%p, seed 42)·97.32%(+3.42%p, seed 43)**로, **CmdI F1**은 **0.9682 → 0.9776(+0.94%p, seed 42)** 및 **0.9618 → 0.9792(+1.74%p, seed 43)**로 향상되었으며(추가 분할인 seed 44·45에서도 CmdI F1 +0.29%p·+0.26%p 개선), 보안 관제 인력이 SQLi 오탐

피로 없이 실제 서버 명령 실행 위험이 있는 **CmdI** 경고를 신뢰하고 즉각 대응할 수 있는 실무적 효과를 제공한다.

2) 선행 연구 설정**(128** 토큰**)** 재구현 비교와 긴 **CmdI** 문맥 잘림 검증**.** 선행 연구(WAMM[10]) 설정인 128 토큰으로 재구현한 DistilBERT는 512 토큰 판본보다 유의하게 낮았으며(seed 42 **0.9718** 대 **0.9914**로 **−1.96%p**, seed 43 **0.9777** 대 **0.9947**로 **−1.70%p**, 둘 다 보정 $p = 0.006$), 심지어 80배 가벼운 char-CNN 단독(0.9884·0.9862)보다도 낮았다. 손실은 128 토큰 상한에서 뒷부분이 잘려 나간 15,553행(6.9%)에 집중되었다(잘린 행 정답률: seed 42 0.994→0.957, seed 43 0.996→0.956; 안 잘린 행은 0.1%p 이내). 특히 주입 구문이 요청 뒷부분에 길게 이어지는 CmdI는 전체의 26.5%가 잘리면서, seed 43에서 CmdI 미탐(CmdI→Normal)이 23건→67건으로 급증해 실제 CmdI 정답 탐지가 4,646건→4,616건으로 줄었고, SQLi→CmdI 오분류도 99→847건(seed 42)·2→551건(seed 43)으로 폭증하여 **CmdI F1**이 **0.9038·0.9284**까지 급락하였다(Table 3). 한편 배치 1에서는 패딩 없이 실제 토큰 길이로 추론하므로 전체 요청의 93.1%가 128 토큰 이하라 추론 지연이 512 토큰과 사실상 같았다(46.10ms 대 46.08ms). 결과적으로 char-CNN → DistilBERT-128 캐스케이드는 DistilBERT-128 단독 대비 합산 **67.5**배 **(F1 +1.66%p, seed 42)·76.3**배 **(+0.84%p, seed 43)**로 두 seed 모두 우위 기준을 통과했으나, 이는 검증 운영점이 사실상 넘김 0%여서 "잘림 없는 **char-CNN** 단독이 **128** 토큰으로 잘린 **DistilBERT**보다 **CmdI** 탐지에 훨씬 정확하고 **76**배 빠르다"는 사실을 반영한 것이다.

3) 템플릿 분할 및 다중 분할 견고성 점검**.** 시험 폴드 요청의 66.5%(CmdI는 14.7%)가 학습 폴드에 같은 골격 템플릿을 두나, 동일 템플릿이 학습과 시험에 나뉘지 않게 묶음 분할(group split)을 수행해도 char-CNN Macro-F1은 +0.10%p(0.9884→0.9894)로 템플릿 암기에 의한 성능 부풀림(사전 기각 기준 −2.0%p)에 해당하지 않았다. 또한 4개 분할 seed(42~45)에서 char-CNN 단독의 Macro-F1 표준편차는 0.18%p(0.9862~0.9901)로 안정적이었으나, 3 에폭 미세조정 DistilBERT의 seed 간 편차(0.9901~0.9947)가 사전 허용폭(0.30%p)과 맞먹어 2차 단독 대비 판정 경계를 갈랐다.

VI. 논의 및 한계

실무 배포 관점의 선택 지침**.** 본 논문의 결과는 인라인 WAF 및 보안 관제 운영 환경에 따라 명확한 설계 기준을 제시한다. 1. 이진 차단**(**정상 대 공격**)** 위주이거나 약 **0.30~0.85%p**의 **Macro-F1** 차이가 허용되는 환경이라면, 무거운 트랜스포머 없이 **char-CNN** 단독 또는 결합 학습된 내부 조기 종료 모델**(**char-CNN-EE**, k3** 보조 출구**)**을 배치하는 것이 512 토큰 DistilBERT보다 **80.2**배**(char-CNN** 단독**, F1** 차이 **−0.30~−0.85%p)** 내지 **284~289**배**(**char-CNN-EE**, F1** 차이 **−0.40~−0.52%p)** 빠르면서 전체 공격 차단율도 거의 차이가 없다. 2. 보안 관제에서 고위험 **CmdI** 경보를 **SQLi**와 명확히 구분해 우선 대응해야 하는 환경이라면, char-CNN → DistilBERT 캐스케이드**(**합산 **10.6~12.3**배 가속**,** 평균 **3.7~4.3ms)** 또는 조기 종료 출구를 1차로 결합한 k3 exit → DistilBERT 캐스케이드**(**합산 **23.8~29.3**배 가속**,** 평균 **1.6~1.9ms)**가 최적의 절충안이 된다. 이 구성은 전체 트래픽의 0.7~0.8%만 2차로 넘기면서도 char-CNN 단독 대비 전체 Macro-F1(+0.28~+0.53%p)과 CmdI F1(+0.94~+1.74%p), CmdI 경보 정밀도(93.9~95.2% → 97.1~97.3%)를 끌어올려, 단일 스레드 CPU에서도 수 밀리초 이내의 지연으로 신뢰성 높은 CmdI 경고를 생성한다.

한계 및 향후 과제**.** 본 연구의 결과는 다음 여섯 가지 한계 안에서 해석되어야 한다. 첫째, 모든 지연은 데스크톱 CPU 1대·단일 스레드·배치 1·PyTorch 기본 구현에서만 측정하였다. 다중 스레드 CPU 추론이나 GPU 배치 서빙에서는 트랜스포머의 병렬화 효율로 1·2차 지연 비율(t_R / t_1)이 줄어 캐스케이드 가속 배수가 작아질 수 있다. 둘째, 교차검증 풀 내 순수 실사용자 정상 요청이 550건(폴드당 약 110건)뿐이어서 실사용자 정상 넘김률 r_N의 폴드 간 분산이 크다(5.2절에서 특정 폴드 배수가 2.95배로 낮아진 원인). 셋째, DistilBERT 재구현은 선행 연구(WAMM[10])의 128 토큰 대신 문맥 잘림이 없는 512 토큰과 Adam을 주 기준으로 사용하였다(원 설정인 128 토큰 판본은 5.3절에서 별도로 대조하여 긴 CmdI 요청의 문맥 잘림 손실을 입증함). 넷째, 1차가 확신도 0.99 이상으로 틀리고 2차가 맞힌 꼬리 오답 110건(대조군 110건)을 원문 감사했을 때 70.9%(대조군 31.8%)가 원본 데이터의 라벨 불일치로 분류되었으나, 이는 대형 언어모델 1인 판정 기준에서 라벨 불일치 가능성이 높았다는 수준이며 보안 전문가 복수 독립 감사는 수행하지

않았다. 다섯째, 비용 기반 지연 유발 공격(slowdown attack)[8] 노출을 검증하기 위해 체크포인트 저장용 재학습 실행(비변형 기준 단일 임계값 합산 배수 23.5배, 클래스별 임계값 45.7배) 위에서 공격 페이로드에 고정 변형을 가해 실측한 결과, 모델 내부 정보 없이 URL 인코딩 1회(url_encode)만 적용해도 1차(char-CNN)의 공격 넘김률 r_A가 단일 임계값 기준 0.27% → 8.76%로 약 **32**배, 공격 요청당 기대 지연이 0.73ms → 4.64ms로 **6.4**배 증가하였다(4개 변형 평균 공격 넘김률 배율은 단일 임계값 ×11.0, 클래스별 임계값 ×2.78). 즉 DeepSloth[8]의 지연 유발 공격이 문제 공간(problem-space) 페이로드 변형만으로 웹 공격 캐스케이드에 전이되며, 2차가 1차보다 약 76배 무거워 넘김 증가가 비용으로 크게 증폭된다. 다만 변형된 공격 요청도 기대 지연 4.64ms로 DistilBERT 단독(46.1ms)보다는 약 10배 빠르며 공격 기저율 3.3% 환경에서 전체 평균 지연 증가는 약 0.13ms에 그친다. 오히려 더 치명적인 한계는 url_encode 적용 시 **1**차 **char-CNN**의 공격 탐지율 자체가 **99.4% → 86.7%**로 떨어졌다는 점이다. 1차가 정상으로 확신한 변형 공격은 2차로 넘어가지 않으므로 그대로 통과되어, 비용 증가가 아니라 **1**차 탐지 회피**(evasion)**가 발생한다(전처리 단계의 반복 URL 디코딩 정규화나 변형 증강 학습이 필수적임). 여섯째, 평가는 단일 데이터셋(SR-BH 2020, 정상 요청의 대다수가 스캐너 탐침)에서 수행되었다.

VII. 결론

본 논문은 인라인 WAF 환경에서 경량 문자 수준 CNN(char-CNN)과 사전학습 트랜스포머(DistilBERT)를 결합한 딥러닝 캐스케이드 및 조기 종료 구조의 속도–정확도 교환과 보안 관제 효용, 그리고 넘김 판단의 구조적 한계를 실트래픽 기반 SR-BH 2020 데이터셋에서 사전 등록 기준에 따라 평가하였다. 실험 결과, char-CNN → DistilBERT 캐스케이드는 시험 요청의 0.7~0.8%만 2차로 넘기면서 char-CNN 단독 대비 전체 Macro-F1(+0.28~+0.53%p)과 소수 고위험 공격인 CmdI F1(+0.94~+1.74%p)을 개선하고 SQLi–CmdI 오분류를 56~74% 줄여 관제 환경에서 신뢰할 수 있는 CmdI 경고를 확보하는 동시에, DistilBERT 단독 대비 합산 **12.3**배(확인 분할 **10.6**배; 결합 학습된 조기 종료 출구를 1차로 결합 시 **23.8~29.3**배)의 추론 가속을 달성하였다. 아울러 선행 연구의 128 토큰 제한이 긴 CmdI 구문을 잘라내어 큰 성능 손실(−1.70~−1.96%p)을 유발함을 규명하였다. 그러나 다중 분할 검증에서 2차 모델 미세조정 편차에 따른 상대 손실 변동(−0.03%p 대 −0.32%p), 단일 임계값의 실사용자 정상 과잉 넘김(특정 폴드 33.6%)과 예측 클래스별 임계값의 재현 실패, 그리고 URL 인코딩 변형에 의한 공격 지연 증가(**6.4**배) 및 1차 탐지 회피(99.4% → 86.7%)가 함께 확인되었다. 결론적으로 웹 공격 탐지 캐스케이드의 성패는 넘김 임계값 조정이 아니라 1·2차 지연 격차와 1차 모델의 판별력 및 입력 정규화 강건성에 달려 있으며, 캐스케이드의 성능 주장은 반드시 종단간 공정 지연, 배포 공격 기저율, 실사용자 정상 넘김률, 다중 분할 분산을 함께 보고할 때에만 성립한다.

References

[1] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," Proceedings of the 31st USENIX Security Symposium, pp. 3971-3988, Aug. 2022.

[2] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," Advances in Neural Information Processing Systems (NIPS), pp. 649-657, Dec. 2015.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, Oct. 2019.

[4] X. Wang, D. Kondratyuk, E. Christiansen, K.M. Kitani, Y. Movshovitz-Attias, and E. Eban, "Wisdom of committees: an overlooked approach to faster and more accurate models," International Conference on Learning Representations (ICLR), Apr. 2022.

[5] S. Teerapittayanon, B. McDanel, and H.T. Kung, "BranchyNet: fast inference via early exiting from deep neural networks," Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp. 2464-2469, Dec. 2016.

[6] K. Tasdemir, R. Khan, F. Siddiqui, S. Sezer, F. Kurugollu, S.B. Yengec-Tasdemir, and A. Bolat, "Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP," arXiv:2312.13041, Dec. 2023.

[7] W. Jitkrittum, N. Gupta, A.K. Menon, H. Narasimhan, A.S. Rawat, and S. Kumar, "When does confidence-based cascade deferral suffice?," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2023.

[8] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitraş, "A panda? No, it's a sloth: slowdown attacks on adaptive multi-exit neural network inference," International Conference on Learning Representations (ICLR), May 2021.

[9] F. Sureda Riera, J. Bermejo Higuera, J.R. Bermejo Higuera, J.J. Martínez Herraiz, and J.A. Sicilia Montalvo, "A new multi-label dataset for web attacks CAPEC classification using machine learning techniques," Computers & Security, 120, 102788, Sep. 2022.

[10] H. Osama, O. Elebiary, Y. Qassim, M. Amgad, A. Maghawry, A. Saafan, and H. Ghalwash, "Enhanced web payload classification using WAMM: an AI-based framework for dataset refinement and model evaluation," arXiv:2512.23610, Dec. 2025.

[11] J.R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," Scientific Reports, 14, 1803, Jan. 2024.

[12] K. Luo, Y. Xie, S. Zhao, and J. Pan, "WADBERT: dual-channel web attack detection based on BERT models," arXiv:2601.21893, Jan. 2026.

[13] M. Bitussi and R. Doriguzzi-Corin, "X-WAD: explainable web anomaly detection," arXiv:2608.27172, Aug. 2026.

[14] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," International Conference on Learning Representations (ICLR), Apr. 2017.

[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 4878-4887, Dec. 2017.

[16] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7076-7087, Jul. 2020.

[17] S. Enomoto and T. Eda, "Learning to cascade: confidence calibration for improving the accuracy and computational cost of cascade inference systems," Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), pp. 7331-7339, Feb. 2021.

[18] C. Guo, G. Pleiss, Y. Sun, and K.Q. Weinberger, "On calibration of modern neural networks," Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321-1330, Aug. 2017.

[19] V.J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J.S. Gardner, I. Hubara, S. Idgunji, T.B. Jablin, J. Jiao, T.S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A.T.R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446-459, May 2020.

[20] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS), pp. 1-7, Nov. 1999.

[21] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, 52(3), pp. 239-281, Sep. 2003.